RETAIL DEMAND FORECASTING OPTIMIZATIONUSING DYNAMIC PROGRAMMING

Design and Analysis of Algorithms (CSA06)CO4 – Industry Problem Solving TaskQ39

REPORT

A practical Dynamic Programming based forecasting solution for retail demand, seasonal trends, demand variability and inventory planning.

Name	T . Kodanda Ram
reg no	192524162
Question	Q39 – Retail Demand Forecasting Optimization (Dynamic Programming)
Problem Domain	Retail / Supply Chain Management
Algorithm	Dynamic Programming
Dataset	180 daily retail-demand records
Implementation	Python, NumPy, Pandas, Matplotlib, Google Colab
Forecast Horizon	14 days
Evaluation	MAE, RMSE, MAPE and moving-average baseline

This report is structured to address the Q39 requirements: industry problem understanding, engineering concept application, solution design, implementation challenges, computational complexity and interpretation in supply-chain optimization. The assignment asks the student to analyze how Dynamic Programming can model demand prediction while considering seasonal trends and data variability.

1. Introduction

Retail companies must maintain enough inventory to satisfy customer demand without holding excessive stock. If demand is underestimated, stock-outs and lost sales may occur. If demand is overestimated, the company may carry unnecessary inventory, increasing holding and storage costs. Demand forecasting therefore acts as an important decision-support component in inventory and supply-chain management.

The selected problem is Retail Demand Forecasting Optimization using Dynamic Programming. The assignment specifically requires consideration of seasonal trends and data variability, followed by a forecasting design, implementation discussion, computational-complexity evaluation and interpretation in supply-chain optimization.

1.1 Problem Statement

The objective is to estimate future retail product demand from historical observations and use the forecast to support inventory planning. The proposed system receives a CSV dataset, preprocesses the demand series, identifies weekly seasonality, represents demand using discrete forecast states, and uses Dynamic Programming to choose an optimized sequence of states.

1.2 Objectives

Accept a retail-demand dataset through Google Colab file upload.

Clean and organize historical demand observations.

Identify recurring weekly seasonal patterns.

Measure recent demand variability.

Represent demand as a finite set of forecast states.

Use a Dynamic Programming recurrence to minimize cumulative forecasting and transition costs.

Evaluate predictions on unseen test observations using MAE, RMSE and MAPE.

Compare the DP-based forecast with a 7-day moving-average baseline.

Generate a 14-day future forecast and an inventory recommendation.

1.3 Industry Context

In a retail supply chain, forecasts can support replenishment planning, purchasing, warehouse preparation and safety-stock decisions. The proposed implementation is intentionally lightweight and suitable for an academic demonstration, while the same architecture can be extended with product-level, promotion, price, holiday and lead-time information.

2. Industry Problem Analysis and Constraints

2.1 Demand Variability

Retail demand is rarely constant. Customer preferences, promotions, holidays, weekday effects and random fluctuations can produce changes from one period to another. The implementation measures short-term variability through the standard deviation of recent demand observations. This variability contributes a penalty to the DP cost so that unstable demand is explicitly represented.

2.2 Seasonal Trends

The dataset contains daily observations. The implementation models a weekly seasonal pattern using the average demand associated with each day of the week. A seasonal factor is calculated as the weekday average divided by the overall trainingaverage. These factors adjust the expected demand according to the day being forecast.

2.3 Data Constraints

Missing or invalid dates must be handled before forecasting.

Missing demand values can distort averages and state construction.

Duplicate dates should not be allowed to create inconsistent observations.

Demand values should be non-negative.

Forecasting must not use future actual demand when generating a prediction.

The model should remain computationally manageable as the number of observations and forecast states grows.

2.4 Data Leakage Prevention

A key improvement in the final implementation is prevention of data leakage. For a test-day prediction, only demand observations that were available before that day are used. The actual demand of the current test day is added to the history only after its prediction has been generated. The actual value is then used for evaluation. This mirrors a rollingforecasting situation more closely than selecting a forecast after observing the target.

2.5 Input Dataset

Field	Meaning
Date	Daily observation date
Demand	Retail product demand in units
Promotion	Synthetic promotion indicator/amount included in the generated dataset
Day Of Week	Day name
Month	Month number

The forecasting algorithm requires Date and Demand. The additional fields are retained as useful contextual information and can support future extensions.

3. Proposed Solution and Dynamic Programming Design

3.1 System Workflow

The proposed workflow is: Dataset Upload → Preprocessing → Training/Test Split → Seasonal Analysis → Forecast-State Generation → DP Optimization → Test Forecast → Accuracy Evaluation → Future Forecast → Inventory

Recommendation.

3.2 Forecast States

Instead of allowing an unlimited number of possible forecast values, the model creates a finite set of demand states from training-data percentiles. Seven states are used: Very Low, Low, Below Average, Average, Above Average, High and Very High. This discretization makes the optimization problem suitable for a DP state-transition formulation.

State	Interpretation
0	Very Low
1	Low
2	Below Average
3	Average
4	Above Average
5	High
6	Very High

3.3 DP State Representation

Let t represent a time position and k represent a forecast state. DP[t][k] stores the minimum cumulative cost up to time t when state k is selected at time t. The table therefore stores the best value for every time-state combination rather than recomputing the same previous-state decisions repeatedly.

3.4 Recurrence Relation

The recurrence used in the implementation is:

DP[t][k] = Cost(t,k) + min over j { DP[t−1][j] + TransitionPenalty(j,k) }

Here, Cost(t,k) measures the difference between the expected baseline forecast and the candidate state, with a contribution from recent demand variability. TransitionPenalty discourages unrealistic jumps between consecutive demand states.

3.5 Optimal Substructure and Overlapping Subproblems

The formulation has optimal substructure because the best sequence ending in state k at time t can be constructed from the best sequence ending in an appropriate previous state j at time t−1 plus the current transition and forecast costs. It has overlapping subproblems because the same time/state combinations are considered as predecessors for multiple later states. Storing them in the DP table avoids repeated computation.

4. Implementation Methodology

4.1 Tools and Technologies

Tool	Purpose
Python	Core implementation
Google Colab	Execution and interactive dataset upload
Pandas	CSV loading, cleaning and tabular processing
NumPy	Numerical calculations and DP arrays
Matplotlib	Demand, forecast and error visualizations
CSV	Input and output data format

4.2 Preprocessing

The program asks the user to upload a CSV file through Google Colab. The uploaded file is detected automatically, so the user does not have to hard-code a filename. Dates are converted to datetime values, demand is converted to numeric values, invalid dates are removed, duplicate dates are handled, missing demand values are interpolated, and negative demand values are clipped.

4.3 Training and Testing

The dataset is divided chronologically into 80% training observations and 20% testing observations. The training data is used to calculate seasonal factors and demand states. The test data remains unseen during model construction and is used for validation.

4.4 Forecasting Cost

For each candidate state, the model calculates a forecast error relative to a baseline estimate and adds a variability component. A transition penalty is then added when moving from the previous state to the candidate state. The DP table stores the lowest cumulative cost

.4.5 Backtracking

After processing all observations, the state with the smallest final cumulative cost is selected. The parent table is then traversed backwards to reconstruct the optimal sequence of states. This is the backtracking stage of the DP solution.

4.6 Implementation Challenges

Designing a DP state that represents a useful forecasting decision.
Selecting a reasonable number of discrete forecast states.
Accounting for seasonality without introducing future information.

Handling demand variability and noisy observations.
Avoiding data leakage during test forecasting.
Maintaining scalability when the number of time periods or states increases.
Converting the optimized state sequence into a practical inventory decision.

CODE

```
# ============================
# 1. IMPORT LIBRARIES
# ============================

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from datetime import timedelta

# Google Colab file upload
from google.colab import files


# ============================
# 2. CONFIGURATION
# ============================

NUM_STATES = 7

TRAIN_RATIO = 0.80

LAMBDA = 0.15

SAFETY_FACTOR = 1.65

FUTURE_DAYS = 14
```

```
# ============================================================
# 3. UPLOAD DATASET
# ============================================================

print("=" * 75)
print("RETAIL DEMAND FORECASTING USING DYNAMIC PROGRAMMING")
print("=" * 75)

print("\nPlease upload your retail demand CSV dataset.")
print("Select the CSV file from your computer.\n")

uploaded = files.upload()


# ============================================================
# 4. CHECK UPLOADED FILE
# ============================================================

if len(uploaded) == 0:

    raise ValueError(
        "No file was uploaded. Please run the cell again "
        "and upload a CSV dataset."
    )


# Get the first uploaded filename
DATA_FILE = list(uploaded.keys())[0]


print("\nFile uploaded successfully:")
print(DATA_FILE)
```

```
    # Previous 7-day standard deviation
    # ------------------------------------------------

    df["Previous7DayStd"] = (
        df["Demand"]
        .shift(1)
        .rolling(
            window=7,
            min_periods=2
        )
        .std()
    )

    # Fill initial missing standard deviation values
    overall_std = (
        df["Demand"].std()
    )

    if pd.isna(overall_std):
        overall_std = 0

    df["Previous7DayStd"] = (
        df["Previous7DayStd"]
        .fillna(overall_std)
    )

    return df


ata = preprocess_data(
    data
```

```
# ============================================
# 46. SAVE OUTPUT FILES
# ============================================

training_results.to_csv(
    "dp_training_results.csv",
    index=False
)

test_results.to_csv(
    "dp_test_results.csv",
    index=False
)

future_forecast.to_csv(
    "dp_future_forecast.csv",
    index=False
)

comparison.to_csv(
    "model_performance_comparison.csv",
    index=False
)


# ============================================
# 47. COMPLEXITY ANALYSIS
# ============================================
```

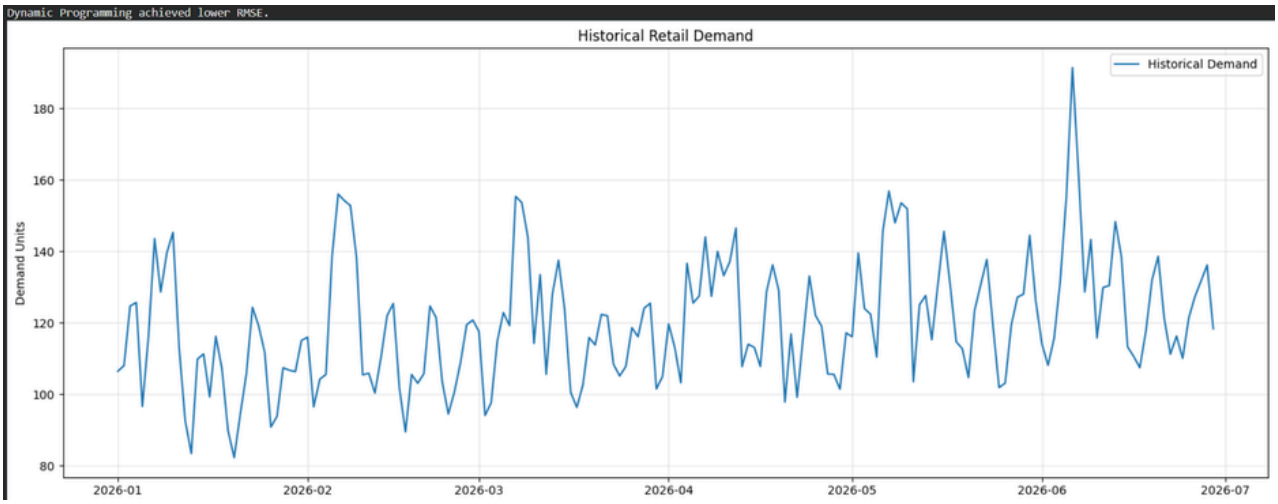
OUTPUT

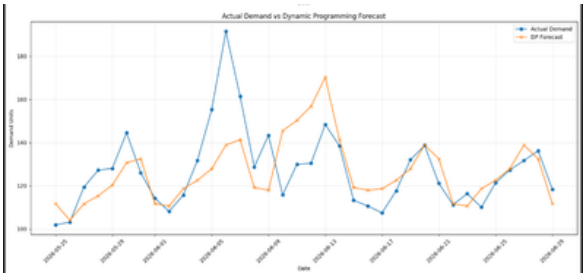
```
================================================================================
ORIGINAL DATASET
================================================================================
         Date  Demand  Promotion  DayOfWeek  Month
0  2026-01-01  106.48       0.00   Thursday      1
1  2026-01-02  108.06       0.00     Friday      1
2  2026-01-03  124.74       0.00   Saturday      1
3  2026-01-04  125.71       0.00     Sunday      1
4  2026-01-05   96.65       0.00     Monday      1
5  2026-01-06  116.24      15.87    Tuesday      1
6  2026-01-07  143.60      27.99  Wednesday      1
7  2026-01-08  128.68      15.31   Thursday      1
8  2026-01-09  139.64      29.55     Friday      1
9  2026-01-10  145.34      17.73   Saturday      1

Number of records: 180

Columns:
['Date', 'Demand', 'Promotion', 'DayOfWeek', 'Month']

================================================================================
PREPROCESSED DATA
================================================================================
         Date  Demand  DayOfWeek  Month  Previous7DayMean  Previous7DayStd
0  2026-01-01  106.48   Thursday      1               NaN        17.393421
1  2026-01-02  108.06     Friday      1        106.480000        17.393421
2  2026-01-03  124.74   Saturday      1        107.270000         1.117229
3  2026-01-04  125.71     Sunday      1        113.093333        10.117200
4  2026-01-05   96.65     Monday      1        116.247500        10.393919
5  2026-01-06  116.24    Tuesday      1        112.328000        12.563342
6  2026-01-07  143.60  Wednesday      1        112.980000        11.349920
7  2026-01-08  128.68   Thursday      1        117.354286        15.533551
8  2026-01-09  139.64     Friday      1        120.525714        15.206156
9  2026-01-10  145.34   Saturday      1        125.037143        15.571631
10 2026-01-11  113.27     Sunday      1        127.980000        17.351028
11 2026-01-12   92.65     Monday      1        126.202857        18.236742
12 2026-01-13   83.46    Tuesday      1        125.631429        19.346015
13 2026-01-14  109.86  Wednesday      1        120.948571        25.107556
14 2026-01-15  111.32   Thursday      1        116.128571        23.200497
```

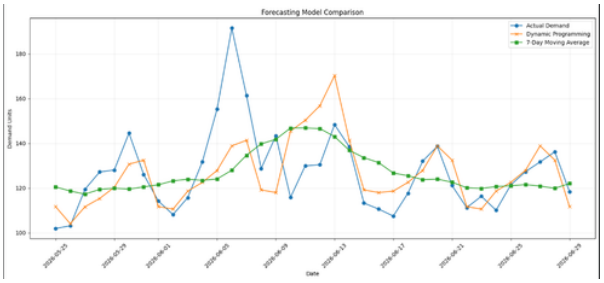
```
================================================================================
DEMAND STATISTICS
================================================================================
count    180.000000
mean     119.951056
std       17.393421
min       82.280000
25%      106.697500
50%      117.680000
75%      130.062500
max      191.480000
Name: Demand, dtype: float64

================================================================================
TRAIN / TEST SPLIT
================================================================================
Total observations : 180
Training records   : 144
Testing records    : 36
Training period    : 2026-01-01 to 2026-05-24
Testing period     : 2026-05-25 to 2026-06-29

================================================================================
WEEKLY SEASONAL FACTORS
================================================================================
Monday     : 0.9049
Tuesday    : 0.8959
Wednesday  : 0.9610
Thursday   : 0.9928
Friday     : 1.0361
Saturday   : 1.1248
Sunday     : 1.0732

================================================================================
FORECAST STATES
================================================================================
State 0 (Very Low     ) : 94.15 units
State 1 (Low          ) : 104.51 units
State 2 (Below Average) : 108.41 units
State 3 (Average      ) : 116.18 units
State 4 (Above Average) : 123.45 units
State 5 (High         ) : 131.71 units
State 6 (Very High    ) : 151.36 units
```

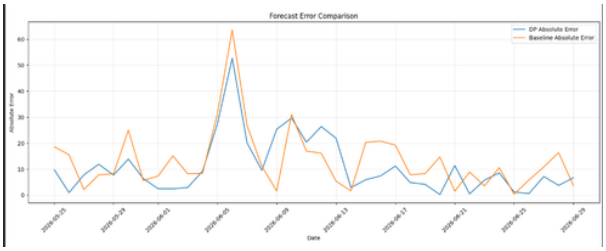
```
================================================================================
DYNAMIC PROGRAMMING TRAINING RESULTS
================================================================================
         Date  Actual_Demand  Baseline_Forecast  DP_Forecast  DP_State
0  2026-01-01         106.48         105.718344       104.51         1
1  2026-01-02         108.06         110.318805       108.41         2
2  2026-01-03         124.74         120.652037       123.45         4
3  2026-01-04         125.71         121.371682       123.45         4
4  2026-01-05          96.65         105.193830       104.51         1
5  2026-01-06         116.24         100.629770       104.51         1
6  2026-01-07         143.60         108.578224       108.41         2
7  2026-01-08         128.68         116.514845       116.18         3
8  2026-01-09         139.64         124.870894       123.45         4
9  2026-01-10         145.34         140.635648       131.71         5
10 2026-01-11         113.27         137.348042       131.71         5
11 2026-01-12          92.65         114.202558       116.18         3
12 2026-01-13          83.46         112.547734       116.18         3
13 2026-01-14         109.86         116.236335       116.18         3
14 2026-01-15         111.32         115.297898       116.18         3
15 2026-01-16          99.28         117.745817       116.18         3
16 2026-01-17         116.27         121.341348       123.45         4
17 2026-01-18         107.23         111.322960       108.41         2
18 2026-01-19          89.85          93.085776        94.15         0
19 2026-01-20          82.28          91.795857        94.15         0

================================================================================
MODEL PERFORMANCE COMPARISON
================================================================================
                Model     MAE    RMSE  MAPE (%)
0  Dynamic Programming  10.844  15.301     8.033
1  7-Day Moving Average 13.309  17.771    10.206
```

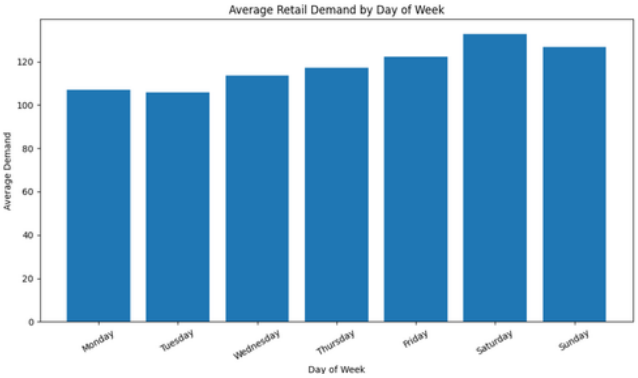
```
================================================================================
TEST FORECAST RESULTS
================================================================================
         Date  Actual_Demand  DP_Forecast  Moving_Average_Forecast  DP_State
0  2026-05-25         101.92   111.711463               120.501429         4
1  2026-05-26         103.15   104.080610               118.668571         3
2  2026-05-27         119.42   111.653550               117.294286         3
3  2026-05-28         127.20   115.348959               119.398571         3
4  2026-05-29         128.08   120.368508               119.930000         3
5  2026-05-30         144.53   130.673568               119.548571         3
6  2026-05-31         126.09   132.486449               120.517143         4
7  2026-06-01         114.17   111.711463               121.484286         4
8  2026-06-02         108.17   110.593487               123.234286         4
9  2026-06-03         115.74   118.640306               123.951429         4
10 2026-06-04         131.81   122.566956               123.425714         4
11 2026-06-05         155.32   127.900605               124.084286         4
12 2026-06-06         191.48   138.850507               127.975714         4
13 2026-06-07         161.44   141.351075               134.682857         5
14 2026-06-08         128.70   119.186041               139.732857         5
15 2026-06-09         143.34   117.993261               141.808571         5
16 2026-06-10         115.83   145.462914               146.832857         6
17 2026-06-11         129.93   150.277315               146.845714         6
18 2026-06-12         130.46   156.816814               146.577143         6
19 2026-06-13         148.37   170.242307               143.025714         6

================================================================================
PERFORMANCE INTERPRETATION
================================================================================
Dynamic Programming achieved lower MAE than the moving-average baseline.
Dynamic Programming achieved lower RMSE.
```

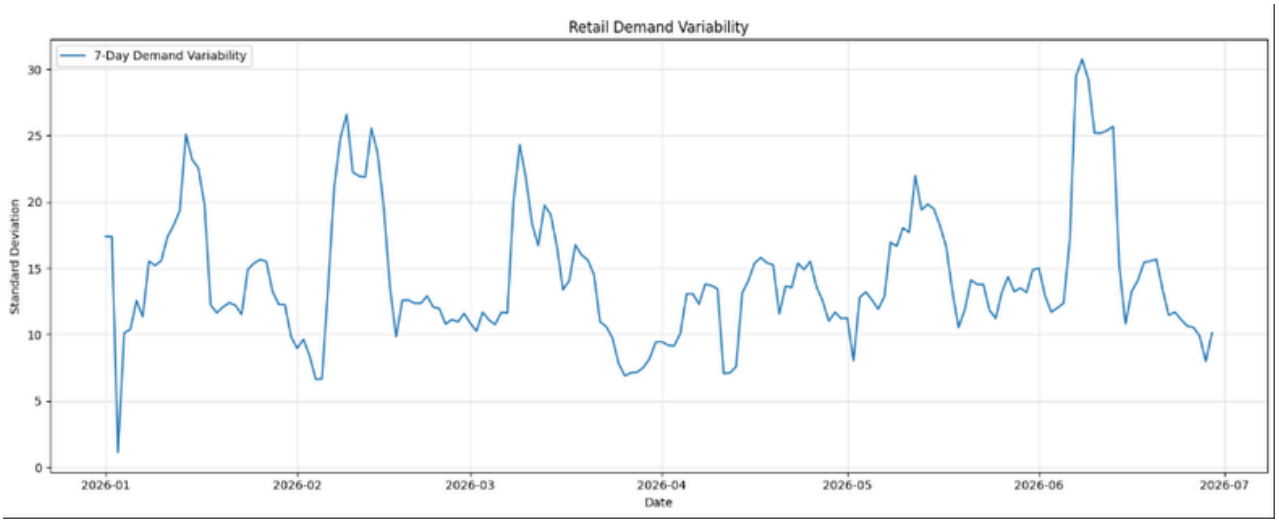






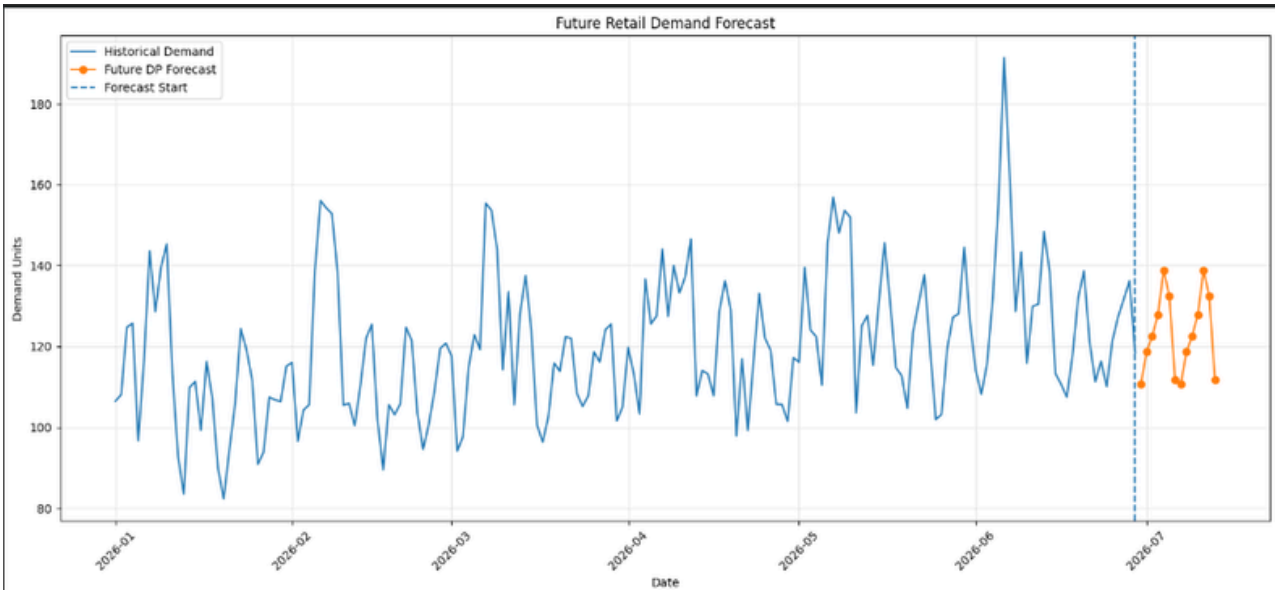







```
================================================================
NEXT 14 DAYS DEMAND FORECAST
================================================================
         Date  Forecast  State     State_Name  Seasonal_Factor  \
0  2026-06-30    110.59      4  Above Average           0.8959
1  2026-07-01    118.64      4  Above Average           0.9610
2  2026-07-02    122.57      4  Above Average           0.9928
3  2026-07-03    127.90      4  Above Average           1.0361
4  2026-07-04    138.85      4  Above Average           1.1248
5  2026-07-05    132.49      4  Above Average           1.0732
6  2026-07-06    111.71      4  Above Average           0.9049
7  2026-07-07    110.59      4  Above Average           0.8959
8  2026-07-08    118.64      4  Above Average           0.9610
9  2026-07-09    122.57      4  Above Average           0.9928
10 2026-07-10    127.90      4  Above Average           1.0361
11 2026-07-11    138.85      4  Above Average           1.1248
12 2026-07-12    132.49      4  Above Average           1.0732
13 2026-07-13    111.71      4  Above Average           0.9049
```

	Baseline_Forecast
0	110.25
1	117.48
2	122.58
3	128.09
4	139.15
5	133.87
6	112.39
7	110.41
8	118.45
9	122.37
10	127.69
11	138.63
12	132.27
13	111.53




```
============================================================
SUPPLY-CHAIN INVENTORY RECOMMENDATION
============================================================
Average future demand : 123.25 units
Recent demand variation : 18.32 units
Safety stock : 30.23 units
Recommended inventory : 153.48 units

============================================================
COMPUTATIONAL COMPLEXITY
============================================================
Number of training observations (n): 144
Number of forecast states (k): 7

Time Complexity: O(n × k²)
Approximate DP state comparisons: 7056

Space Complexity: O(n × k)
DP table dimensions: 144 × 7

============================================================
FINAL PROJECT SUMMARY
============================================================

The Retail Demand Forecasting system:

1. Accepts a CSV dataset through Google Colab upload.

2. Cleans and preprocesses historical retail demand data.

3. Identifies weekly seasonal demand patterns.

4. Measures recent demand variability.

5. Generates multiple forecast states.

6. Uses Dynamic Programming to find an optimized sequence
   of forecast states.

7. Uses optimal substructure and overlapping subproblems.

8. Uses backtracking to reconstruct the optimal state sequence.

9. Generates test forecasts without using the current
   future actual demand.

10. Evaluates the forecast using MAE, RMSE and MAPE.

11. Compares Dynamic Programming with a 7-day moving average.
```

```
6. Uses Dynamic Programming to find an optimized sequence
   of forecast states.

7. Uses optimal substructure and overlapping subproblems.

8. Uses backtracking to reconstruct the optimal state sequence

9. Generates test forecasts without using the current
   future actual demand.

10. Evaluates the forecast using MAE, RMSE and MAPE.

11. Compares Dynamic Programming with a 7-day moving average.

12. Generates a 14-day future demand forecast.

13. Calculates a supply-chain inventory recommendation.

14. Saves the forecasting results as CSV files.

15. Complexity:
       Time  = O(n × k²)
       Space = O(n × k)


============================================================
FINAL PERFORMANCE RESULTS
============================================================

Dynamic Programming
MAE  : 10.844
RMSE : 15.301
MAPE : 8.033%

7-Day Moving Average
MAE  : 13.309
RMSE : 17.771
MAPE : 10.206%

============================================================
OUTPUT FILES CREATED
============================================================
1. dp_training_results.csv
2. dp_test_results.csv
3. dp_future_forecast.csv
4. model_performance_comparison.csv

Program completed successfully!
============================================================
```

5. Results and Performance Evaluation

The following results were calculated using the generated 180-day dataset and the final implementation logic. The dataset contains 144 training observations and 36 test observations.

Metric	Dynamic Programming	7-Day Moving Average
MAE	10.844	13.309
RMSE	15.301	17.771
MAPE	8.033%	10.206%

MAE represents the average absolute forecasting error. RMSE gives greater weight to larger errors. MAPE expresses the average error as a percentage of actual demand. Lower values indicate better predictive accuracy for these measures.

5.1 Interpretation of Validation

For this generated dataset, the DP-based model has a lower MAE than the moving-average baseline.

The DP model also has a lower RMSE, indicating better performance under the squared-error measure.

5.2 Example Test Forecast Records

Date	Actual	DP Forecast	MA Forecast	DP State
2026-05-25	101.92	111.71	120.50	4
2026-05-26	103.15	104.08	118.67	3
2026-05-27	119.42	111.65	117.29	3
2026-05-28	127.20	115.35	119.40	3
2026-05-29	128.08	120.37	119.93	3
2026-05-30	144.53	130.67	119.55	3
2026-05-31	126.09	132.49	120.52	4
2026-06-01	114.17	111.71	121.48	4

2026-06-02	108.17	110.59	123.23	4
2026-06-03	115.74	118.64	123.95	4

5.3 Why Validation Matters

A forecasting system should not be judged only by the generated forecast graph. Quantitative validation provides evidence about prediction error. The comparison with a simple moving-average baseline also provides a reference point, helping determine whether the additional DP formulation provides useful performance for the chosen dataset .

6. Future Forecast and Supply-Chain Interpretation

6.1 Fourteen-Day Forecast

Future Date	Forecast Units	State	Seasonal Factor
2026-06-30	110.59	4	0.896
2026-07-01	118.64	4	0.961
2026-07-02	122.57	4	0.993
2026-07-03	127.90	4	1.036
2026-07-04	138.85	4	1.125
2026-07-05	132.49	4	1.073
2026-07-06	111.71	4	0.905
2026-07-07	110.59	4	0.896
2026-07-08	118.64	4	0.961
2026-07-09	122.57	4	0.993
2026-07-10	127.90	4	1.036
2026-07-11	138.85	4	1.125
2026-07-12	132.49	4	1.073
2026-07-13	111.71	4	0.905

6.2 Inventory Recommendation

The implementation uses a simple safety-stock calculation to connect the forecast to supply-chain decision making. The average future forecast is combined with a safety-stock allowance based on recent

demand standard deviation.

Safety Stock = Safety Factor × Recent Demand Standard Deviation

Recommended Inventory = Average Future Forecast + Safety Stock

Inventory Measure	Calculated Value
Average 14-day future demand	123.25 units
Recent 30-day standard deviation	18.32 units
Safety factor	1.65
Safety stock	30.23 units
Recommended inventory	153.48 units

6.3 Supply-Chain Meaning

Procurement teams can use the forecast to plan replenishment quantities.

Warehouse managers can prepare capacity for periods with higher predicted demand.

Inventory planners can use safety stock to reduce the risk of stock-outs caused by demand variation.

Seasonal factors help identify recurring weekly demand patterns.

The forecast can support decisions about purchasing and stock positioning, although actual business decisions would also require lead time, supplier capacity, service-level targets and storage constraints.

6.4 Scalability

The main DP computation uses n time positions and k forecast states. For every time position and current state, the implementation examines possible previous states. Therefore, the time complexity is $O(nk^2)$, while storing the complete DP and parent tables requires $O(nk)$ space.

7. Complexity, Limitations and Conclusion

7.1 Computational Complexity

Component	Complexity
Forecast-state generation	O(n log n) approximately due to percentile computation
DP optimization	$O(n \times k^2)$
DP table storage	O(n × k)
Test rolling prediction	O(m × k), where m is test length
Future recursive forecast	O(h × k), where h is forecast horizon

For the selected implementation, n is the number of training observations and k is the number of forecast states. With 144 training records and 7 states, the DP portion remains small enough for interactive execution in Google Colab. In a larger retail environment, state count, product count and forecasting frequency would become important scalability considerations.

7.2 Limitations

The model uses a discrete set of demand states rather than continuous probabilistic forecasts.

Weekly seasonality is represented primarily through day-of-week factors.

The generated academic dataset is synthetic; real retail data may contain stronger promotions, holidays, product interactions and missing observations.

The inventory recommendation is a simplified safety-stock calculation and does not model lead time, service-level targets or supplier constraints.

Dynamic Programming optimizes the defined state-transition cost; the quality of the final forecast depends on the state representation and cost function.

7.3 Future Enhancements

Add product-level forecasting and multiple SKUs.

Include holidays, promotions, price and external demand drivers.

Introduce lead-time and storage-capacity constraints directly into the optimization.

Use rolling retraining as new demand observations arrive.

Compare the DP formulation with ARIMA, exponential smoothing or machine-learning models.

Deploy the forecasting component as an API or dashboard for supply-chain planners.

7.4 Conclusion

The proposed Retail Demand Forecasting Optimization system demonstrates how Dynamic

Programming can be formulated for a practical demand-prediction problem. Historical demand is converted into a finite set of forecast states, and DP selects an optimized sequence by minimizing cumulative forecasting and transition costs. Seasonal trends and demand variability are explicitly incorporated.

The implementation also addresses a critical forecasting concern: data leakage. Predictions for unseen test days are generated from information available before the prediction, while actual demand is used only for subsequent rolling updates and evaluation. MAE, RMSE and MAPE provide quantitative validation, and comparison with a 7-day moving average establishes a simple benchmark.

From a supply-chain perspective, the resulting forecast can support inventory planning, replenishment and safety-stock decisions. The solution is computationally manageable for the demonstration dataset, with $O(nk^2)$ time and $O(nk)$ space complexity. Overall, the project provides a feasible academic demonstration of Dynamic Programming applied to retail demand forecasting and inventory optimization.

8. References / Basis

1. SIMATS Engineering, Assessment Tool 2, Design and Analysis of Algorithms (CSA06), CO4 – Industry Problem Solving Task, Q39: Retail Demand Forecasting Optimization (Dynamic Programming).

2. Project implementation and generated dataset: retail_demand_dataset.csv; Python implementation using Pandas, NumPy and Matplotlib in Google Colab.

3. Evaluation measures used in the implementation: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and Mean Absolute Percentage Error (MAPE).